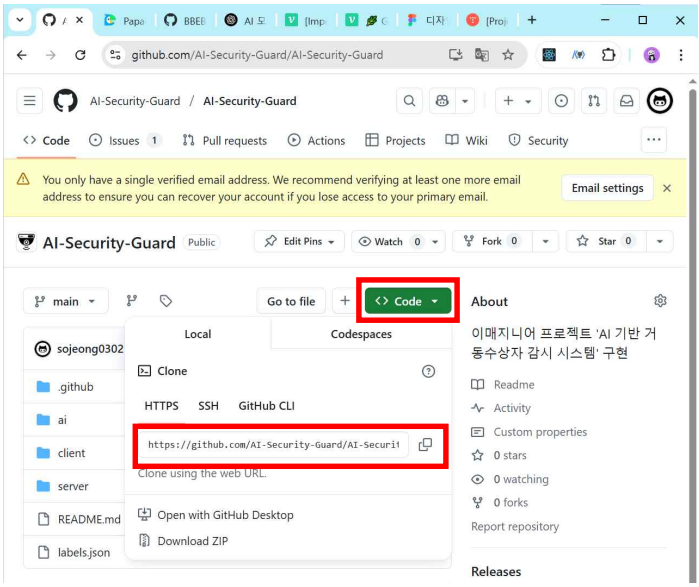
※ 이 글은 깃허브 데스크탑 기준으로 작성되었습니다.

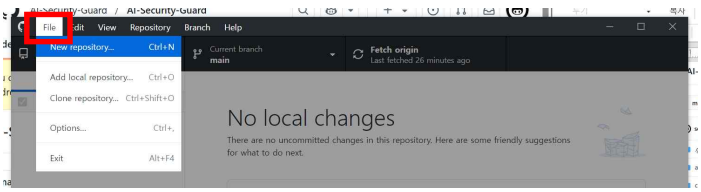
1. 저장소 가져오기(클론)

클론이란 쉽게 말해서 복제한다는 건데
저희는 깃허브 저장소를 복제해서 작업을 해줘야합니다.

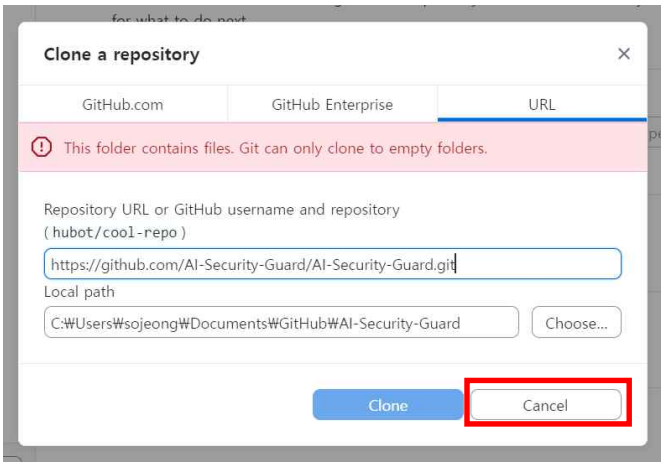
깃허브 저장소에 가서 code클릭하면 있는 url 복사하고, 깃허브 데스크탑으로 가주세요



깃허브 데스크탑 file에 가면 Clone repository가 있습니다.



거기에 URL 들어가서 아까 복사한 url 넣어주고 Clone 클릭하면 됩니다.



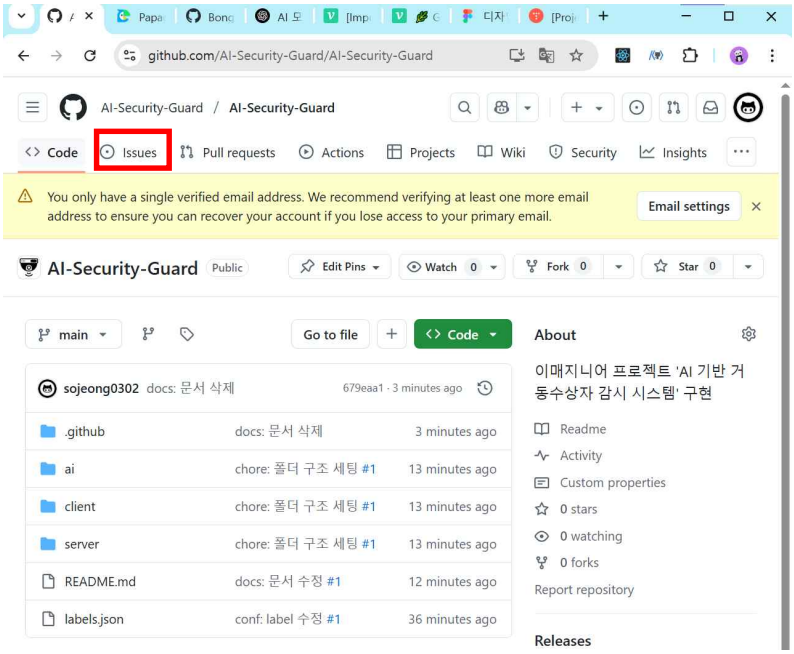
2. 이슈 생성

이슈는 쉽게 말해서 본인이 해야 할 작업을 적어놓는 노트인데
여러명이서 프로젝트를 할 때에 혼란을 방지하기 위해 꼭 필요한 작업입니다.

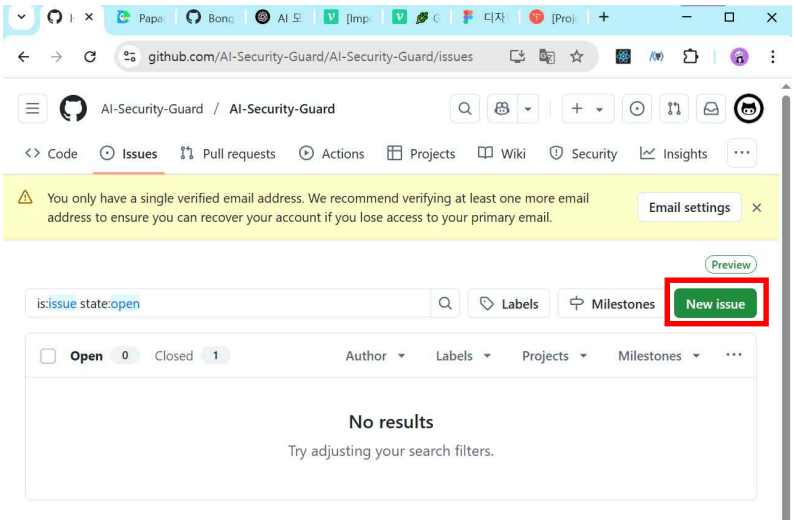
> A: 나 로그인 구현 해야겠다.
> B: 그거 내가 이미 구현했던 건데?
> A: 아 이미 절반이나 구현했는데...

서로 소통을 하더라도 한계가 있으니까 기록하는 게 좋겠죠?

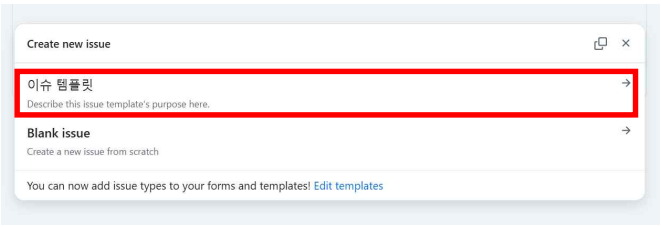
깃허브 저장소 들어가서 두번째 Issues 클릭해주세요.



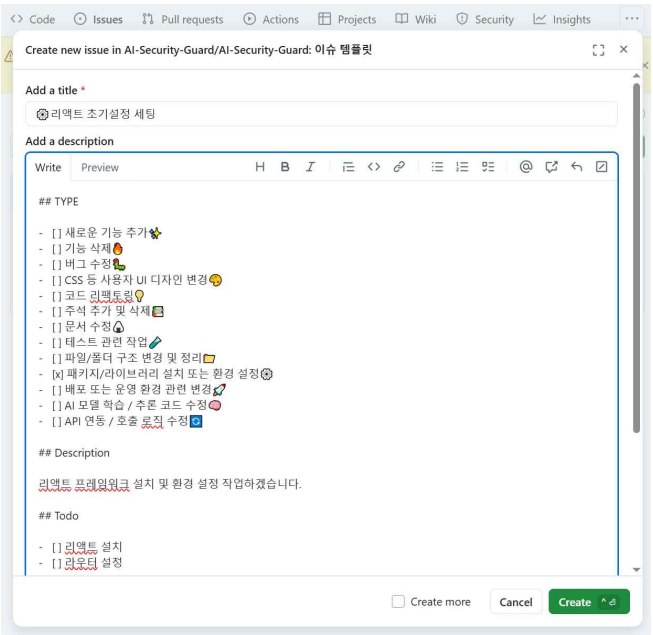
New issue 클릭해주세요.



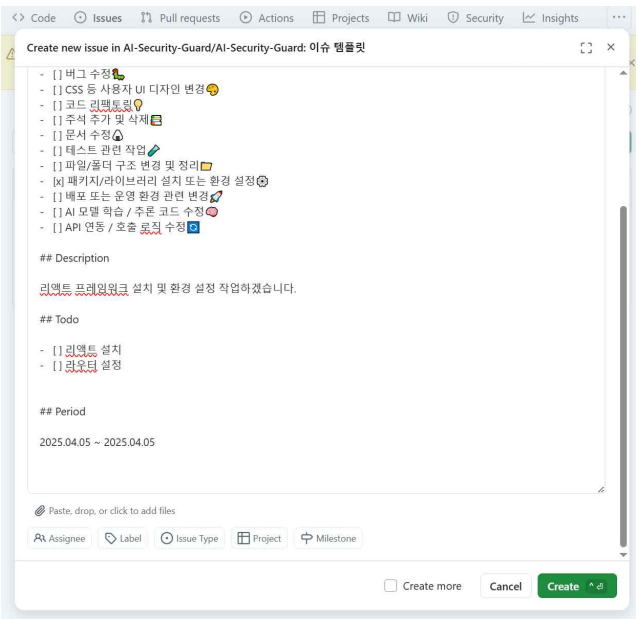
그럼 아래 사진처럼 창이 한개 나오는데, 여기서 첫 번째 이슈 템플릿 클릭해주세요.



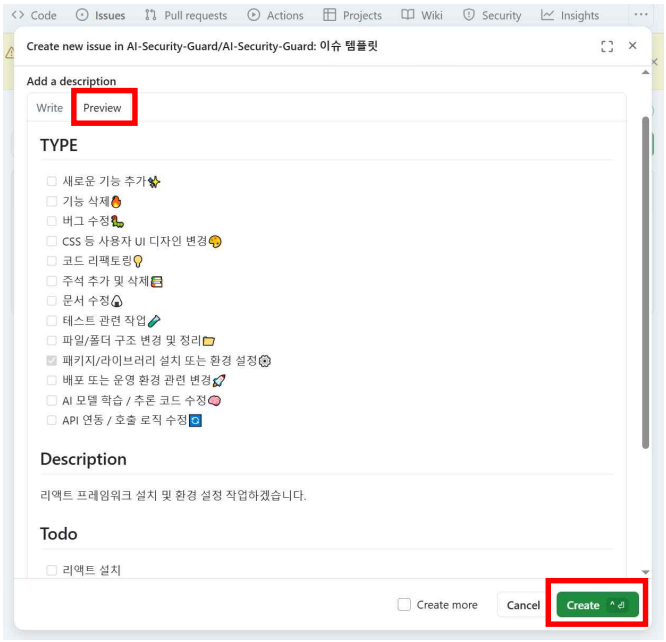
제가 만들어놓은 템플릿인데 작업하려는 타입을 파악해서 type에 [x]로 표시하고 아이콘 title 앞에 붙여주세요. (title명은 되도록 직관적으로 작성해주세요)



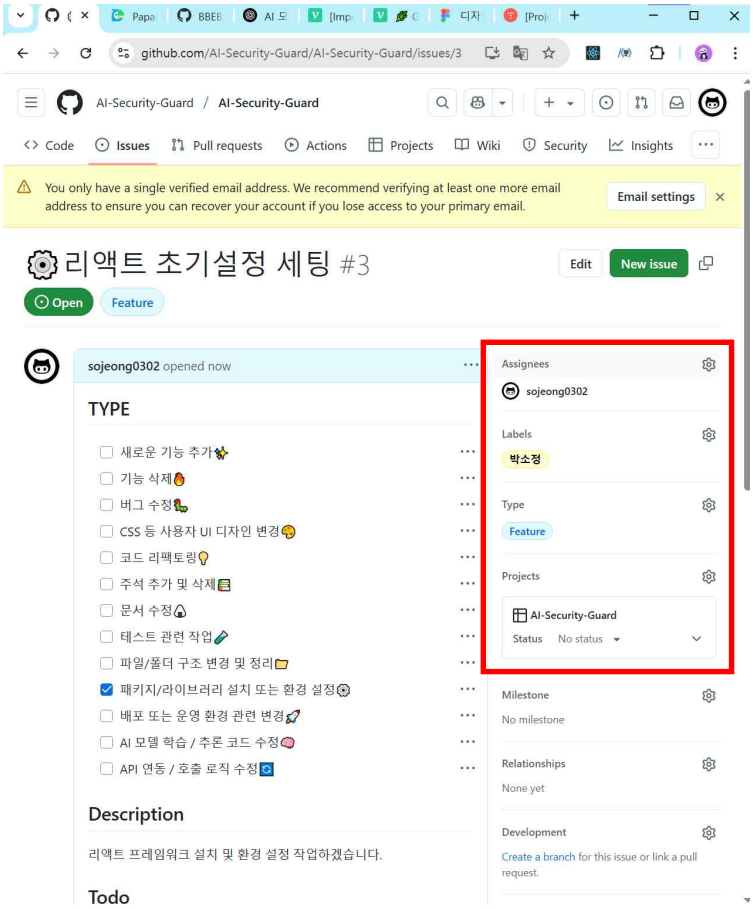
나머지 내용도 작성해주세요.



마크다운 문서가 잘 못 작성됐을 수도 있으니까 Preview로 확인하고 Create 클릭해주세요.



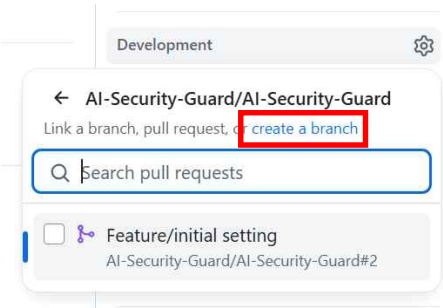
이슈생성하면 아래 사진처럼 이슈 화면이 나옵니다.
오른쪽에 Assignees, Labels, Type, Projects도 지정해주세요.



3. 브랜치 생성

아까 이슈가 할 일 적어놓는 노트같은거라고 했죠?
브랜치는 일(이슈)을 할 수 있는 작업 공간을 뜻합니다.
main에서 바로 작업을 할 수는 없으니까 브랜치를 생성을 해야 합니다.

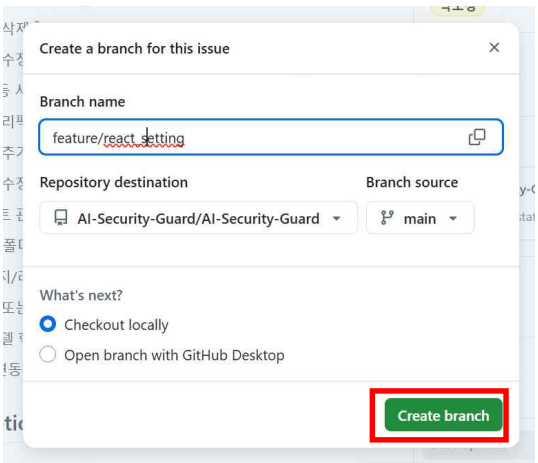
이슈 화면 오른쪽 하단에 Development가 있습니다.
클릭하면 아래 사진처럼 나오는데 여기서 create a branch를 클릭해줍니다.



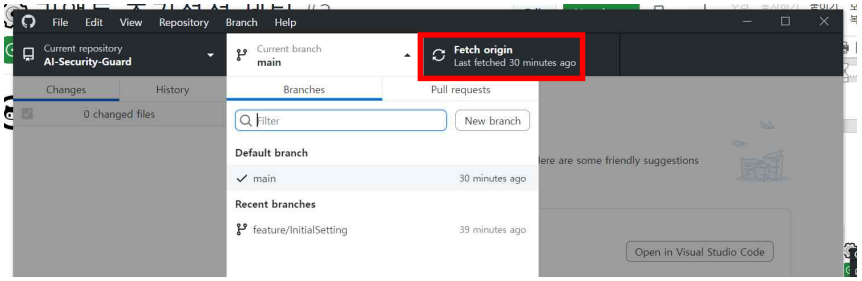
그럼 브랜치를 생성할 수 있는 창이 나옵니다.
[type]/[브랜치 이름]으로 작성해주세요.
type은 이슈 작성할 때 지정해놓은 거 있죠?

```
feat: 새로운 기능 추가
remove: 기능 삭제
fix: 버그 수정
style: CSS 등 사용자 UI 디자인 변경
refactor: 코드 리팩토링
comment:: 주석 추가 및 삭제
docs: 문서 수정
test: 테스트 관련 작업
file: 파일/폴더 구조 변경 및 정리
conf: 패키지/라이브러리 설치 또는 환경 설정
ai: AI 모델 학습 / 추론 코드 수정
api: API 연동 / 호출 로직 수정
```

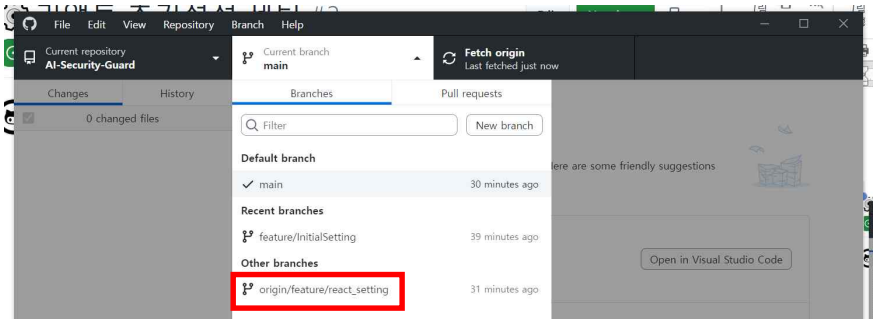
여기서 찾아서 적어 준 뒤에 create branch 해주세요.
(화면상에는 feat가 아니라 feature로 되어있는데, 저도 앞으로 feat로 통일하겠습니다...)



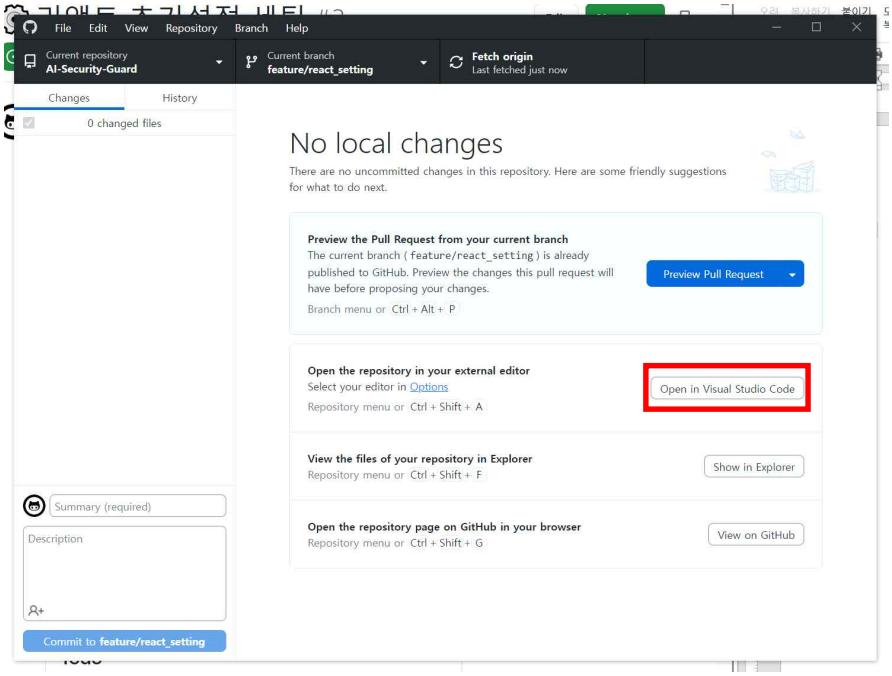
브랜치를 main에서 방금 만든 브랜치로 변경해서 작업해야하는데
깃허브 데스크탑 들어가면 아래 사진처럼 만들어놨던 브랜치가 안 보입니다.
당황하지 말고 바로 오른쪽에 있는 Fetch origin을 클릭해주세요.



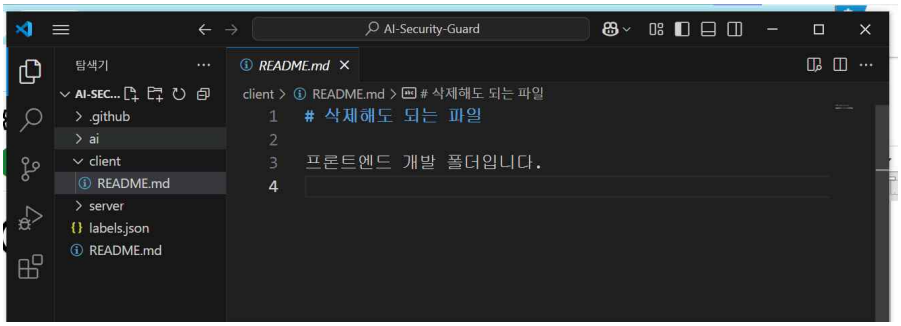
이제 생성해놨던 브랜치가 보일 겁니다.



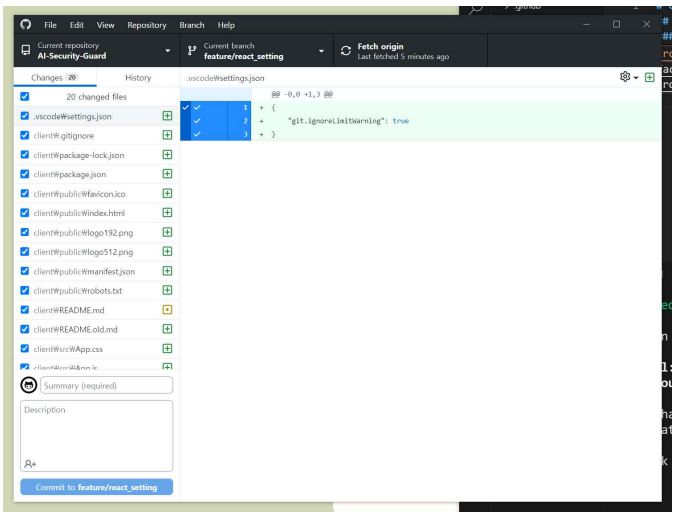
브랜치가 main이 아닌 본인이 만들어놨던 브랜치인걸 꼭꼭꼭 확인해주시고
open in vs code 클릭해주세요
(main에서 바로 작업하면 안됩니다.)



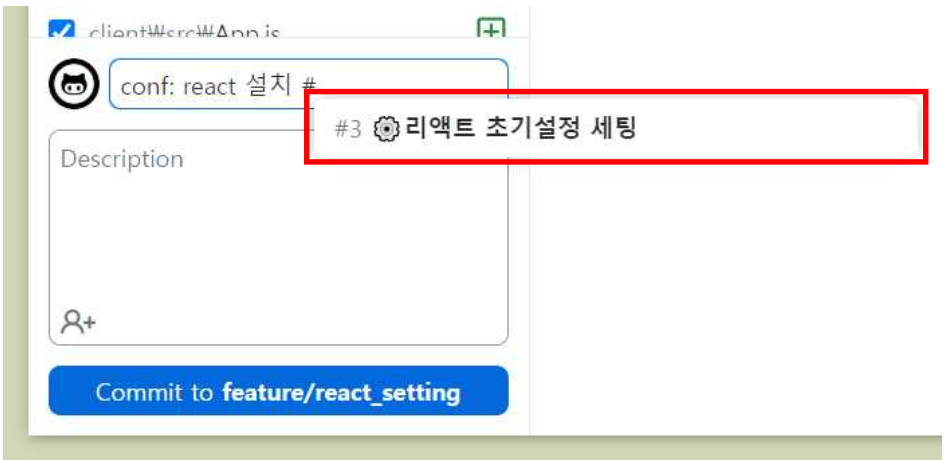
그럼 작업할 수 있는 환경이 나옵니다. 이제 여기서 작업해주세요!



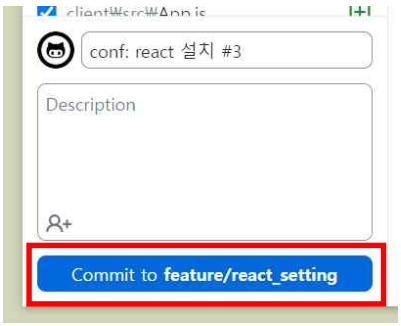
작업하다가 깃허브 데스크탑을 보시면 변경사항을 알 수 있습니다.
여기서 이 내용을 커밋하고 싶다면, 왼쪽 하단에 커밋 메시지를 작성해야 합니다.



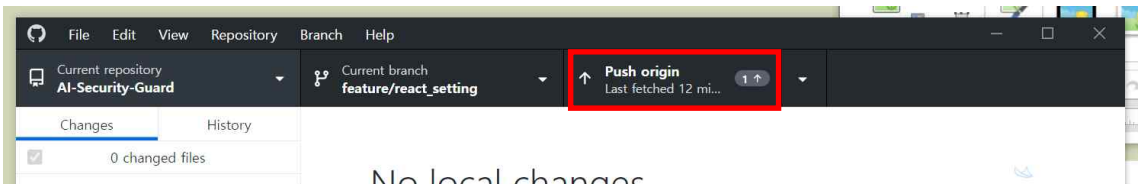
커밋 메시지는 [type]:[커밋 메시지] #이슈번호로 작성하면 되는데
type는 아까 브랜치 생성할 때 사용했던 거 그대로 쓰면 됩니다.
마찬가지로 커밋 메시지는 직관적이게 작성하고
더 자세히 적고 싶으면 밑에 Description 부분에 작성해주세요.
이슈 번호는 #만 적으면 아래 사진처럼 나옵니다.



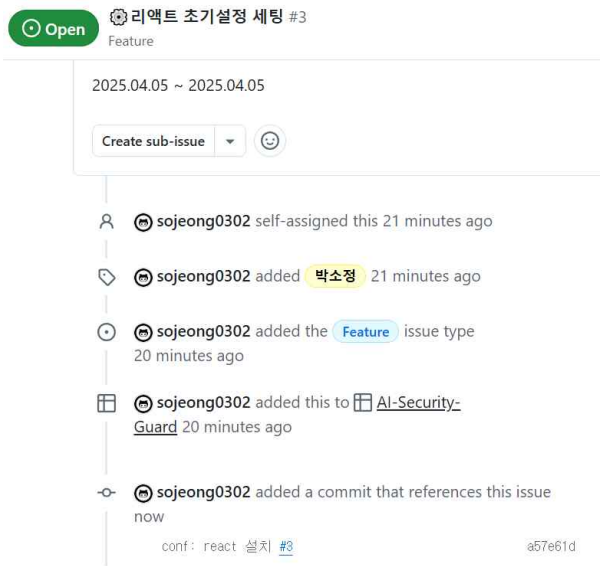
커밋 메시지 작성이 완료됐으면 아래 파란색 버튼을 클릭해주세요



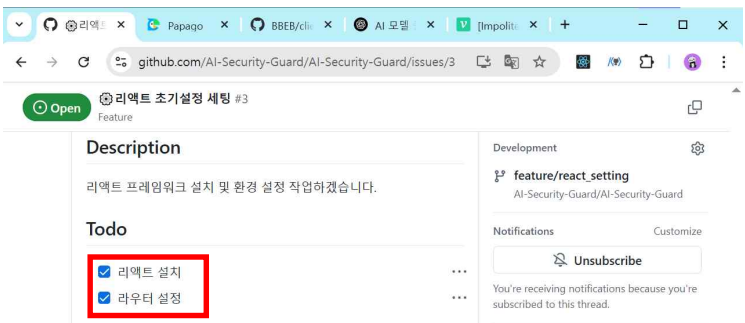
그리고 Push origin을 클릭해주면 커밋이 완료됩니다.



이슈 생성했던 페이지 가보면 작성한 커밋 메시지가 보입니다.



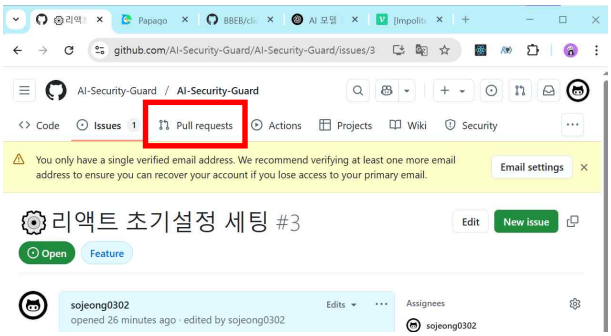
작업을 전부 완료했으면 이슈에서 만들었던 todo에 체크 해줍니다.



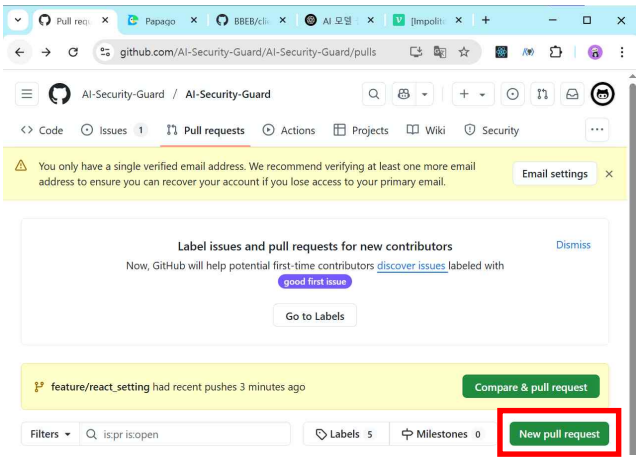
4. Pull requests

이제 pr 요청 작업을 할 건데 쉽게 말해서 담당자한테
"저 이런 작업했는데 확인해주세요!" 라고 요청하는 신청서(?)라고 생각하면 됩니다.
아직 여러분의 코드는 main에 합쳐지지 않았거든요.

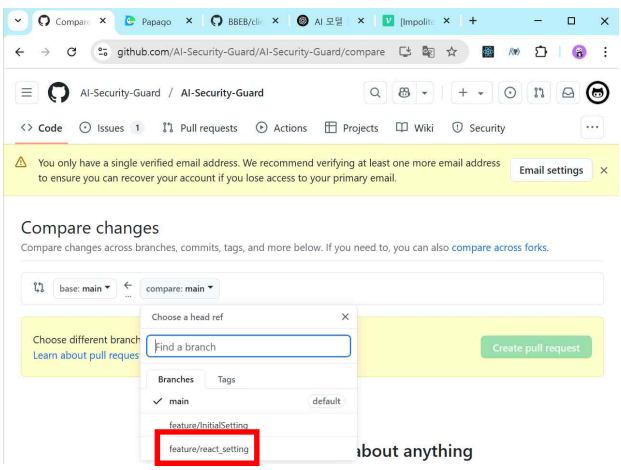
3번째에 있는 pull requests를 클릭해주세요



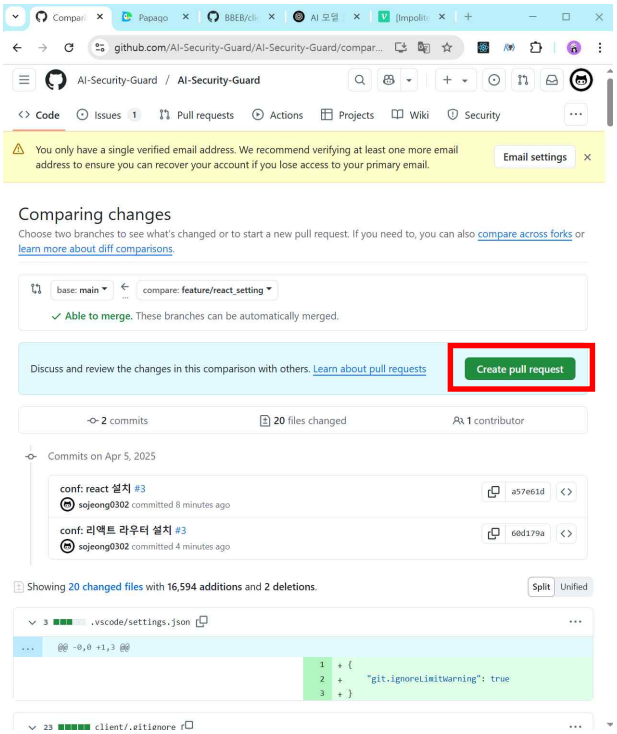
여기서 New pull request를 클릭해주세요.



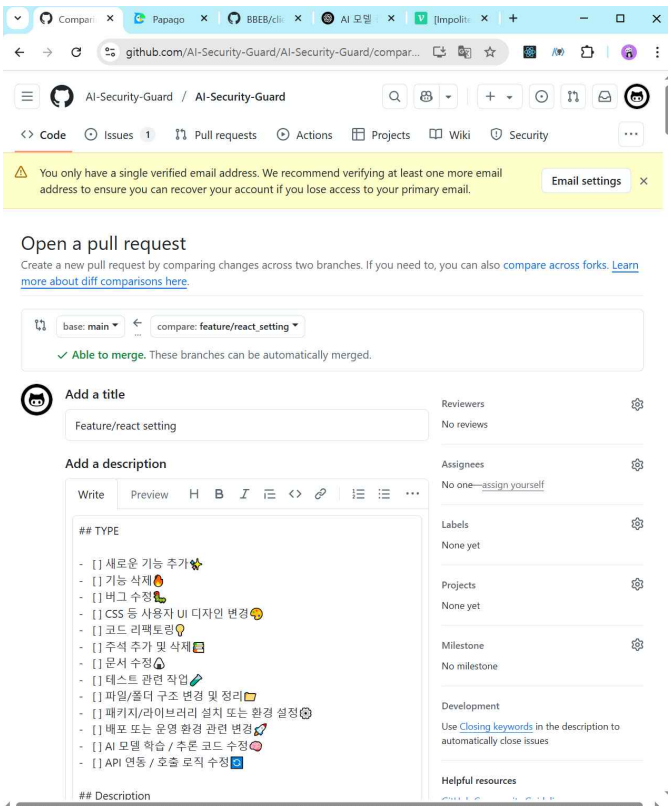
main <- [내 브랜치] 즉, 내가 작업했던 브랜치를 main에 합치겠다는 뜻입니다.
따라서 오른쪽에 내가 작업했던 브랜치를 선택해줍니다.



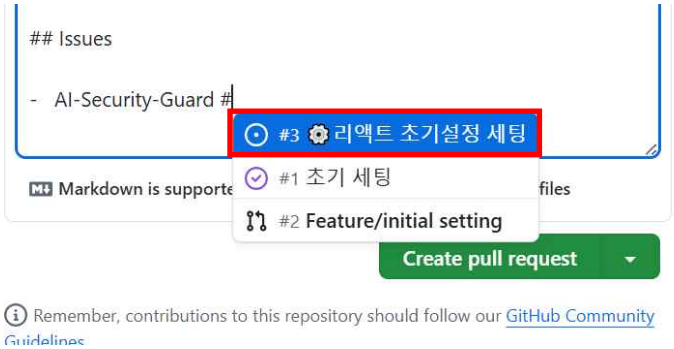
선택하면 작업했던 내용들을 스크롤해서 볼 수 있습니다.
본인이 작업했던 내용인거 확인했으면 Create pull request 클릭해주세요.



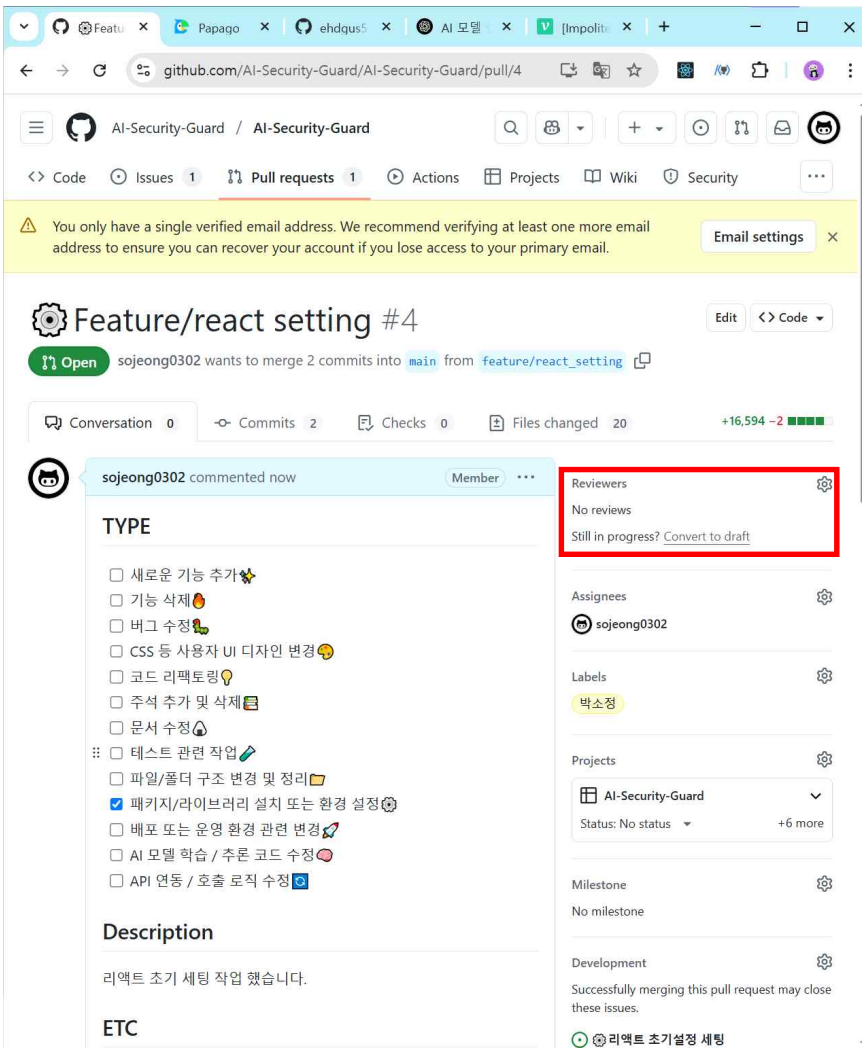
그럼 이슈 만들었을 때처럼 템플릿이 보이는데 마찬가지로 작성해주세요



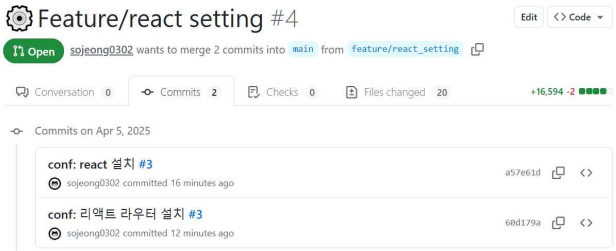
맨 마지막엔 #있으면 이슈 목록들 보여주니까
여기서 본인이 생성한 이슈를 선택 해주면 됩니다.



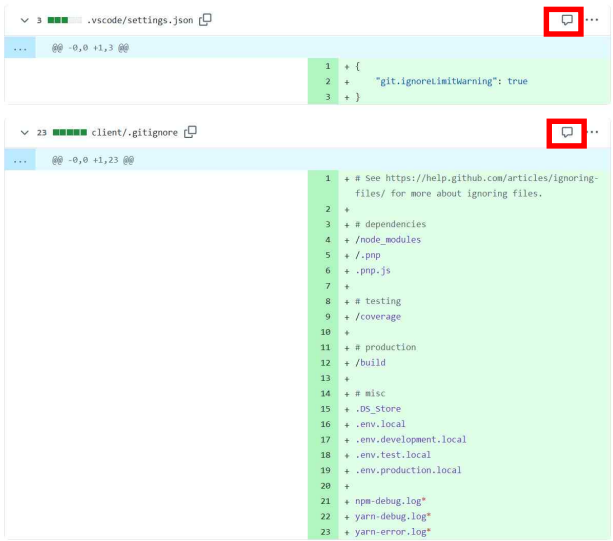
오른쪽에 Reviewers에는 본인이 작업했던 내용을 리뷰 해줬으면 하는 사람 선택 하는 건데 AI개발 쪽은 AI 개발하는 사람 지정하고
웹 개발 쪽은 웹 개발하는 사람 지정하는 게 효율적일 것 같습니다.
그리고 지정하면 아마 메일로 알람이 갔던 것 같은데 메일 확인 못할 수도 있을 것 같으니까 저 링크를 복사해서 카톡으로 보내는 식으로 진행하면 좋을 것 같습니다.



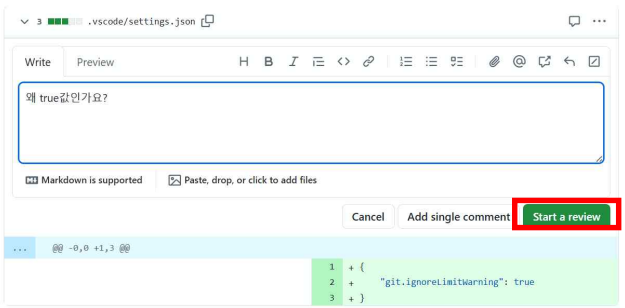
PR보면 commits에 커밋 내역이랑 변경한 코드들도 볼 수 있습니다.



말풍선 모양의 아이콘이 있는데 이걸 통해서 코드 리뷰라는 것을 할 수 있습니다.



Reviewers가 의문인 점이 있다거나, 이 부분에서는 다른 방식으로 코딩했으면 좋겠다거나, 변수명이 좀 저 직관적이었으면 좋겠다거나 등등등 적어 주는 건데 pr 요청한 사람은 이 코드 리뷰를 보면서 수정하면 됩니다.

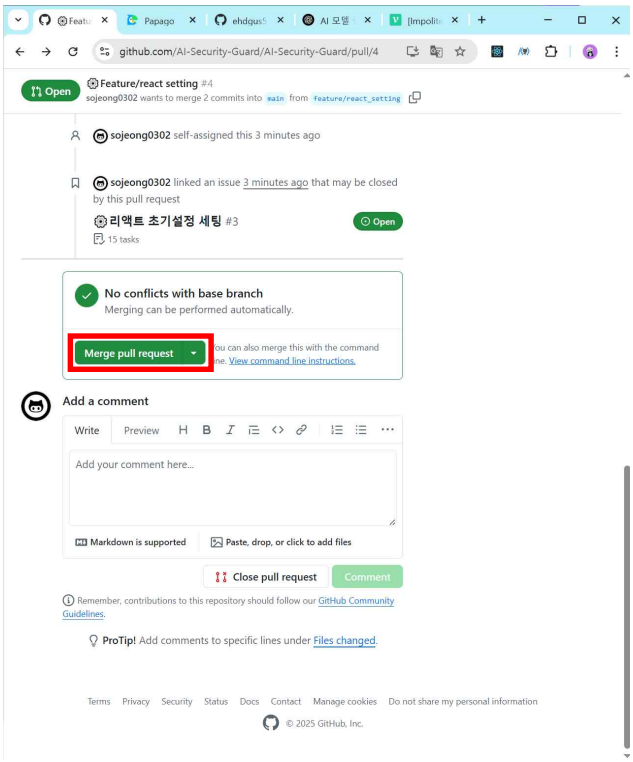




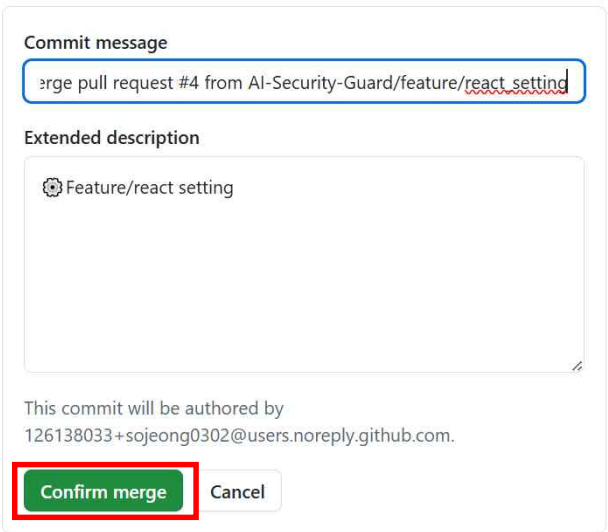
5. 머지

이제 코드에 이상이 없다? 이거 main 코드에 포함시켜도 되겠다 싶을 때 코드를 합쳐주는데 이걸 Merge(머지)라고 합니다.

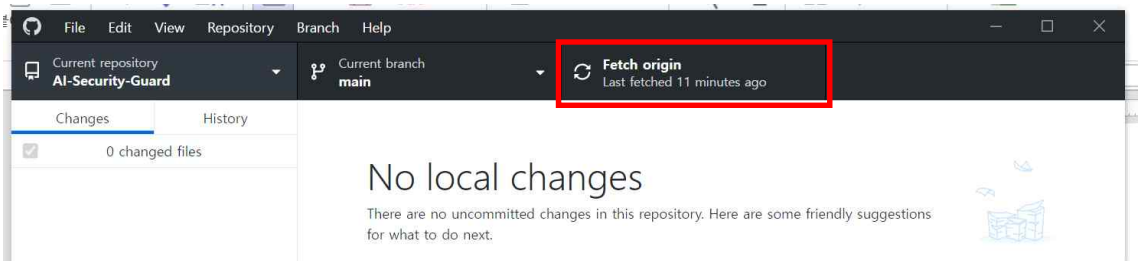
아래 사진처럼 pr요청 페이지에서 스크롤 내리면 Merge pull request가 있는데 그거 클릭해주면 됩니다.



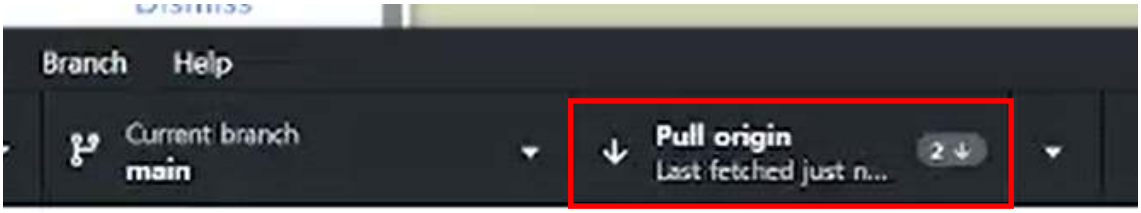
confirm merge 클릭해주면 최종적으로 코드가 합쳐진 건데
보통 이 작업은 pm이 해주는데 저희는 pm이 없으니까 본인이 하던가 아니면 pr 요청할 때 지정된 Reviewers가 해주면 될 것 같습니다.



깃허브 데스크탑에 가서 브랜치를main으로 바꿔주고 Fetch를 클릭해주세요.

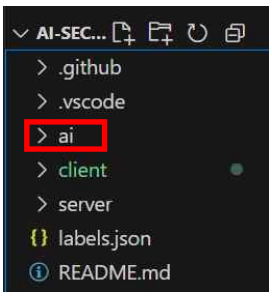


그럼 main에서 변경된 내용들을 내려 받을 수 있습니다.



끝!!

+ 꼭 지켜줬으면 하는 부분
작업 폴더 침범하지 않아야 합니다.
예를 들어서 AI 개발자 분들은 ai 폴더에서 작업해야합니다.



vs code 열어보면 터미널 경로가 아래와 같이 되어있습니다.
(클론 위치에 따라 조금 다를 수 있음)

```
PS C:\Users\sojeong\Documents\GitHub\AI-Security-Guard>
```

이 경로를 ai에서 작업한다고 가정할 때, cd ai를 해주어야합니다.
패키지같은 거 설치할 경우 꼭 경로를 지켜서 설치해주세요!

```
PS C:\Users\sojeong\Documents\GitHub\AI-Security-Guard> cd ai
PS C:\Users\sojeong\Documents\GitHub\AI-Security-Guard\ai>
```

웹은 프론트엔드 작업할 경우엔 cd client, 백엔드 작업할 경우엔 cd server하면 됩니다!!
추가로 이해 안 되는 부분 있으면 언제든 연락주세요.